

FPGA Lab- Theoretical Background

1. Digital logic Recap

1.1. Binary Representation

Given an N -bit long binary string $a[N - 1:0]$, we denote the integer value, represented by this binary string by **angle brackets** $\langle a[N - 1:0] \rangle$. Namely:

$$\langle a[N - 1:0] \rangle \triangleq \sum_{i=0}^{N-1} a[i] \cdot 2^i$$

Example: Take $N = 4$, $a[3:0] = 1110$, then the integer value $\langle a[3:0] \rangle$ equals 14.

Recall that to represent some natural number $x \in \mathbb{N}$ as a binary string, this string would require $N \geq \lceil \log_2 x \rceil$ bits. Conversely, it can be stated that any N bit binary string can represent natural numbers from 0 to $2^N - 1$ inclusive.

1.2. Combinational circuits

A combinational circuit consists of logical gates and nets that connect them together. A combinational circuit must have at least one input and at least one output. Each output corresponds to a single Boolean function, implemented by the circuit. This is basically it – the combinational circuits are implementations of Boolean formulas. They do not have memory that holds some internal state, only a relation between the inputs and the outputs.

A timing analysis of a combinational circuit is finding its critical path, i.e. the longest delay from an input to an output.

1.3. Conditional Sum Adder (CSA)

A combinational circuit $\text{CSA}(n)$ is a commonly known implementation of a binary adder. Hence it respects the following specifications:

Inputs:	$A[n - 1:0], B[n - 1:0] \in \{0:1\}^n, \quad C[0] \in \{0:1\}.$
Output:	$S[n - 1:0] \in \{0:1\}^n, C[n] \in \{0:1\}.$
Functionality:	$\langle S[n - 1:0] \rangle + 2^n C[n] = \langle A[n - 1:0] \rangle + \langle B[n - 1:0] \rangle + C[0]$

The parameter n determines the bit-width of the operands and of the sum. The implementation can be defined recursively with $\text{CSA}(1)$ being a full adder, and for $n > 1$ the design is shown in Figure 1. The idea behind the recursive design is to split the computation in 2 (by choosing k equal roughly to $n/2$) and such that the lower k bits of the operands are processed in parallel to the higher $n - k$ bits of the operands. However, there is a dependence of the computation of the higher $n - k$ bits on the carry generated by the lower bits (carry signal $c[k]$). Hence the name “conditional” in the CSA abbreviation. To overcome this dependence, the higher $n - k$ bits are computed twice, corresponding to two different assumptions $c[k] = 0$ and $c[k] = 1$. Finally, the correctly assumed computation path is chosen (mux) using the $c[k]$ bit from the lower k bits.

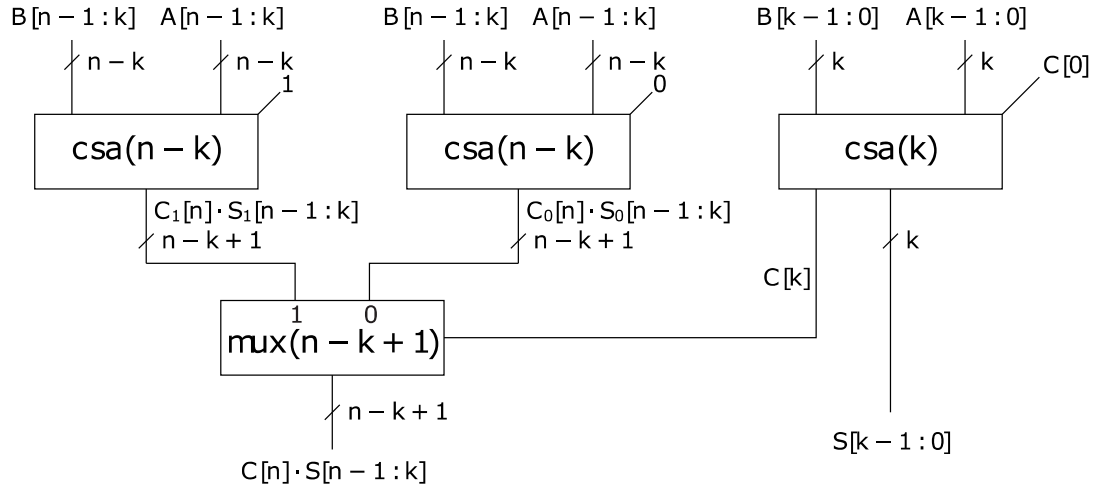


Figure 1 - CSA(n) recursive logical design for $n > 1$. The dot operator stands for a concatenation.

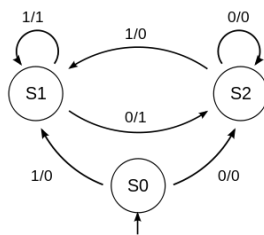
1.4. Synchronous circuits

A synchronous circuit consists of a combinational circuit and flip-flops (FFs). Every synchronous circuit must have a CLOCK input (and in practice also a reset input) to synchronize the operation of the flip-flops over time. A synchronous circuit can be viewed as combinational circuit that is re-run every clock cycle and that some of the information can be stored (in FFs) between the clock cycles. It introduces a notion of a discrete time (i.e. clock cycle number 1, clock cycle number 2, ...) which doesn't exist in combinational circuits.

1.5. Finite State Machines

Every finites state machine (FSM) can be described by a 6-tuple that consists of the set of states, initial state, input alphabet, output alphabet, transition function $\delta(q, in)$ which determines next state as a function of a current state and a current input and output function $\lambda(q, in)$ which determines the current output as a function of a current state and a current input.

For example, consider the following state diagram:



This diagram represents an FSM with the circles stand for states and the arches stand for transitions. The labels over the arches have the format of "input(t)/output(t)". This FSM has an input alphabet of $\{0,1\}$ and an output alphabet of $\{0,1\}$, the set of states is $\{S0, S1, S2\}$ with the initial state S0. The $\delta(q, in)$ function transits to S1 if "1" is received as an input, and to S2 if "0" is received as an input. The $\lambda(q, in)$ function will output 1 if and only if the current state of the machine is S1.

A common mistake is to think that the output, specified over the diagram edges, stands for the output in the following clock cycle. The true definition is that the output(t) must be provided at the same clock cycle as the input(t).

In digital design, FSMs are frequently designed and synthesized for providing control signals for data-path circuits. An FSM, synthesized into a synchronous circuit, is nothing else but a set of FFs (to store the state) and two combinational circuits: one implements the output function $\lambda(q, in)$, and the other implements the $\delta(q, in)$.

In the theory of computation, a Mealy machine is a finite-state machine whose output values are determined both by its current state and the current inputs. (This is in contrast to a Moore machine, whose output values are determined solely by its current state.)

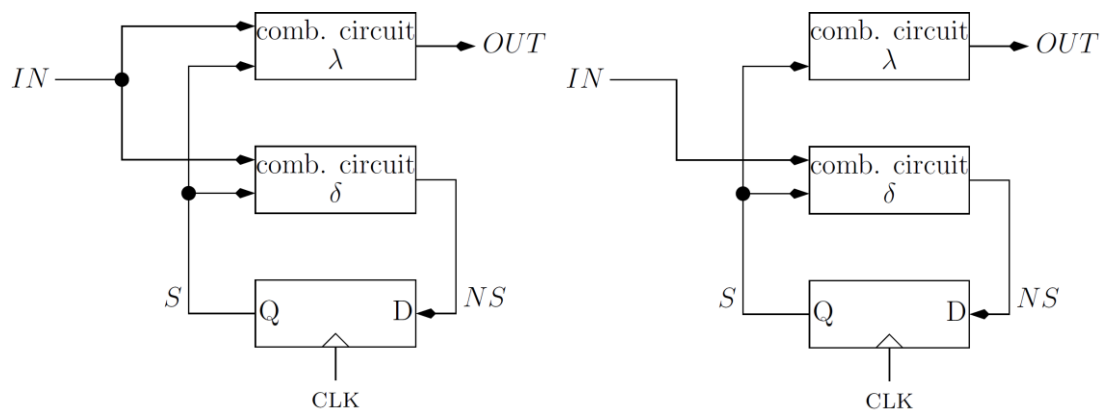


Table 1- A canonical diagram of a Mealey state machine (left) and of the Moore state machine (right)

1.6. Digital Debouncing

The problem of bouncing arises whenever one wants to receive a steady signal from a mechanical switch. Even after the analog signal is quantized to a digital value at every clock cycle, a single press on a button may generate an alternating signal in the first clock cycles after the pressing. A debouncer is a circuit which has an unstable signal input and a stable signal output.

A “counting approach” overcomes this problem in a following manner: a synchronous debouncer-circuit maintains an N-bit saturating¹ counter which is incremented +1 for every clock cycle with a high input and is decremented for every clock cycle with a low input. The stable output will be equal to the MSB of the counter, which effectively ensures that the stable signal (output) will not toggle repeatedly as long as the bouncing effect ends within at most 2^{N-1} clock cycles. See Table 2 (a) for a timing example.

Sometimes, the designer would like to refer to a switching event as a “mono-stable” like event, where a debouncing not only transforms a non-stable input into a stable output, but also clips the stable output to a single clock cycle. See Table 2 (b) for a timing example.

Table 2 - Example of two digital debouncer approaches, both based on a 3-bit saturating counter. Approach (a) asserts stable-output=1 $\leftrightarrow$ there were at least 2^2 more clock cycles with unstable-input=1 than clock cycles with unstable-input=0. Approach (b) asserts a single cycle stable pulse only when the counter value is incremented from 3 to 4 (counter range boundary crossing).

Clock cycle t	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Unstable input	0	1	0	0	1	0	1	1	1	1	1	0	0	0	0

¹ An N bit saturating counter never exceeds the value of 2^N-1 when incremented and never reduces below 0 when decreased.

3-bit Counter Value	0	1	0	0	1	0	1	2	3	4	5	4	3	2	1
(a) Stable output debouncer	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0
(b) Single-cycle output debouncer	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

2. FPGA Technology

Field Programmable Gate Arrays (FPGA) are digital integrated circuits whose functionality can be reconfigured after manufacturing. This is different from ASICs, application specific integrated circuits, which cannot be reconfigured after manufacturing. By “reconfiguring” we mean both changing the functionality of underlying sub-circuits and re-wiring these sub-modules in a different fashion.

2.1. General FPGA structure

FPGA mainly consists of arrays of modules called configurable logic blocks (CLB), surrounded by input-output blocks (IOB). All these blocks are interconnected by a switch matrix which is yet again – reconfigurable, allowing connection from any block to any other block. Figure 2 depicts the structure of an FPGA chip.

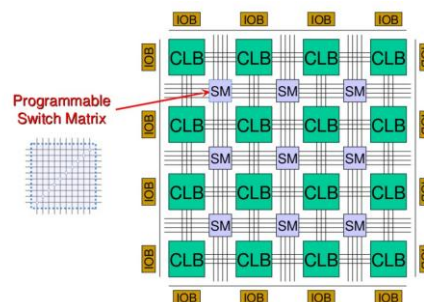


Figure 2 - High Level Diagram of an FPGA

The **IOBs** are very complex modules that can deliver signals between the IO pads of the FPGA chip and the internal signals of the design. IOBs support various slew-rates, single-ended or differential connections. An illustration of an IOB is presented in Figure 3. **CLBs** on the other hand are in charge of implementing small Boolean functions and storing bits in flip-flops. Each CLB contains 2 slices, see Figure 4 for an illustration of a CLB

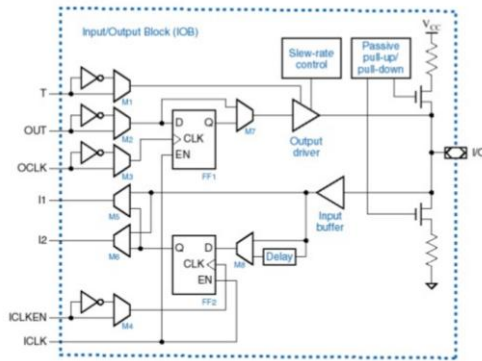


Figure 3 - IOB

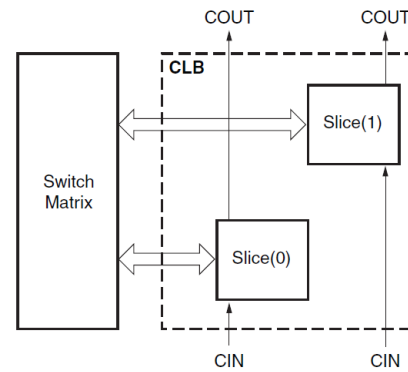


Figure 4 – CLB connected to a switching matrix

Generally, a **slice** is a basic component that implements a configurable logical function and can hold a bit of information in a FF, as depicted in Figure 5. In Xilinx FPGAs, there are 2 types of slices: “SLICEL” which constitutes of two lookup tables and a couple of flip-flops; and “SLICEM” which contains all what SLICEL can offer and adds more memory capabilities to it

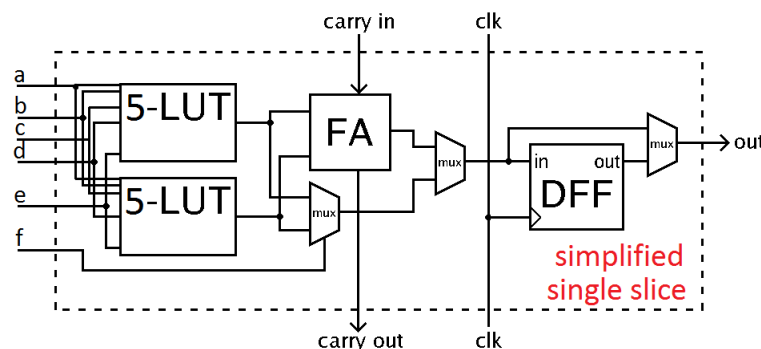


Figure 5 - generic view of a slice

2.2. Additional FPGA blocks

In addition to the CLBs and the IOBs which are the basic building blocks of the FPGA, there are additional blocks that provide special purpose capabilities.

Digital Clock Management (DCM) module provides de-skew, phase shifting, frequency division and other useful clock outputs. Keeping in mind that the clock signal is pure, and synchronized signal, we cannot simply generate multiple clocks using logic gates. In fact, it is a very bad practice. Instead, the DCM modules should be used to produce clean clock signals when multiple clock domains must exist on a chip. If no multiple domains are required, then the single clock is pulled from the external oscillator and distributed in the FPGA through a *global buffer* (BUFG), a high fan-out distribution wiring in the FPGA chip.

Digital Signal Processing (DSP) slices provide high performance multiplication and addition operations. Since these operations are very common in almost any application, the FPGA vendors tend to create more and more DSP slices on their high end FPGAs. This of course comes at a cost of wasting potential area for more CLBs, but it implements multiplications and additions using much less area than it would take the CLBs to be configured into an equivalent circuitry.

Block RAM (BRAM) are dedicated modules for storing data on chip. Could be thought of as a sliceM, but with all the LUTs stripped away from it and having more registers added to it.

3. FPGA Design Flow

This section explains a high-level scheme of how one implements a design on FPGA. This design flow consists of multiple stages which include (i) RTL design (ii) Design elaboration and simulation, (ii) Synthesis, (ii) Implementation and Device programming

3.1. RTL Design

Hardware description languages (HDL) can describe the hardware without explicitly defining all the logical circuit structure. This level of design is commonly referred to as Register Transfer Level (RTL). This level of design differs from Gate-Level (GL), where a logical structure of the circuit is described at the resolution of logical gates and netlists. We use languages such as Verilog or VHDL to design the full RTL functionality of the digital modules in our system. **After the RTL design stage, we end up with a set of source files which fully describe our system functionality.**

3.2. RTL Analysis

First the Verilog compiler performs **Static Elaboration** of the design code, which reminds the pre-processor stage of the C code compiler. During this stage, the parameters and the generate-blocks are resolved into a solid block diagram, which we call “**Elaborated design**”.

Behavioral **simulation** scenario is intended for a verification of a logical correctness of the elaborated design. Simulation scenarios are defined using a test-bench source file (also written in HDL) which examines the logical functionality of the design by performing the following:

1. Instantiating an HDL unit under test (UUT)
2. Driving the UUT with stimulus (inputs signals)
3. Comparing the outputs of the UUT vs expected values

We perform behavioral simulation during which we can observe the internal signals of the UUT in a form of a waveform diagrams (In the document “Vivado_Tutorial” you can read about simulation in Vivado software). This way we identify wrong and unexpected signals which do not correspond to the expected functionality. It is crucial to perform simulations of our design, since many design bugs can be easily and conveniently eliminated during this stage. Dealing with the malfunctions in later design stages will be more painful.

3.3. Synthesis

The synthesizer program converts HDL (VHDL/Verilog) code into a gate-level netlist. represented in the terms of the UNISIM component library - a Xilinx library containing basic primitives. Xilinx design environment uses built-in synthesizer XST (Xilinx Synthesis Technology). Other synthesizers can also be used. XST output is a netlist stored in NGC format. (there is also EDIF for netlists)

Synthesis report provides useful information. There is a maximum frequency estimate in the "timing summary" chapter. One should also pay attention to warnings since they can indicate hidden problems.

3.4. Implementation

Implementation stage is intended to translate netlist into the placed and routed FPGA design. Xilinx implementation stage is divided into 3 sub-stages: logical optimization

(opt_design), placement (place_design) and routing (route_design). These steps are specific for Xilinx, whereas Altera combines opt_design into one step executed by quartus_map.

Logical optimization - performed by the opt_design program. Logic optimization ensures the most efficient logic design before attempting placement. It performs a netlist connectivity check to warn of potential design problems such as nets with multiple drivers and un-driven inputs. Logic optimization also performs block RAM power optimization.

Placement and routing - Placement is performed by place_design program and routing is performed by the route_design program. Place and route is the most important and time-consuming step of the implementation. It defines how device resources are located and interconnected inside an FPGA.

Placement is even more important than routing, because bad placement would make good routing impossible. To provide possibility for FPGA designers to tweak placement, PAR has a "starting cost table" option (sort of a seed of the random placement generator).

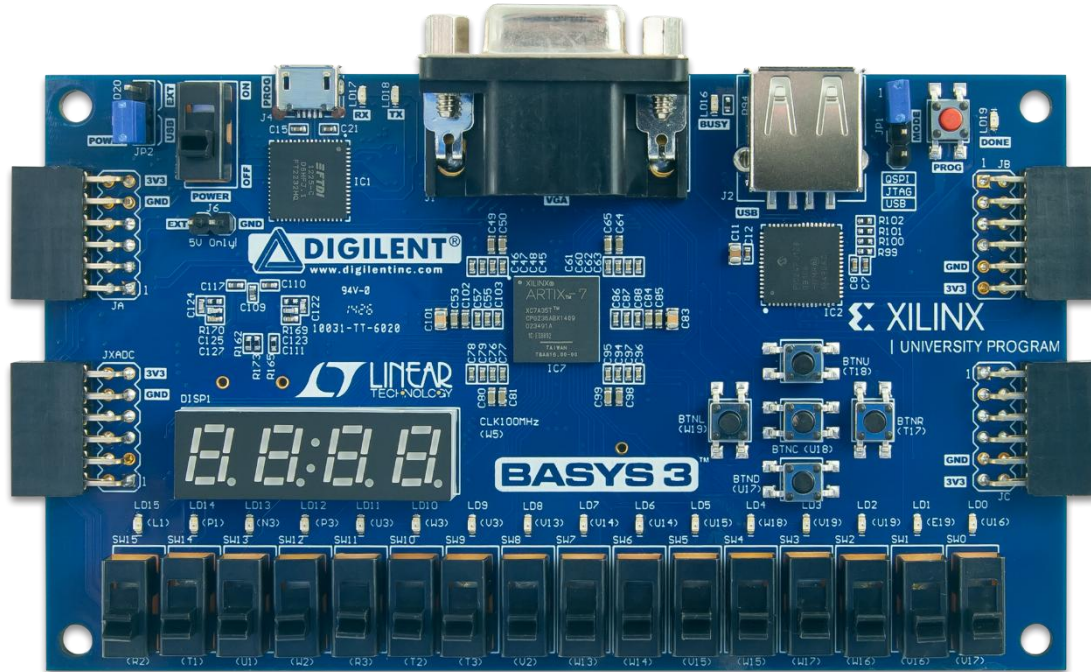
Placement and routing accounts for timing constraints set up by the FPGA designer. If at least one constraint can't be met, PAR returns an error.

3.5. Device Programming

The products of the implemented design are formed in to a *.bit file which can be streamed to the FPGA and configure all its CLBs, IOBs and other components. In the document "Vivado_Tutorial" you can read about device programming in Vivado environment.

4. Lab Hardware

The hardware platform for our FPGA lab would be BASYS3 board with Artix-7 FPGA chip. A good reference for its specifications can be found [here](#). Specifically, you need to get familiar with the [basic IO](#) section and a [7-segment](#) subsection.



Artix-7 FPGA (Xilinx part number XC7A35T-1CPG236C) includes:

- 33,280 logic cells in 5200 slices (each slice contains four 6-input LUTs and 8 flip-flops)
- 1,800 Kbits of fast block RAM
- Five clock management tiles, each with a phase-locked loop (PLL)
- 90 DSP slices
- Internal clock speeds exceeding 450MHz
- On-chip analog-to-digital converter (XADC).

The Basys3 also offers a collection of ports and peripherals, including:

- 16 user switches (SW0-SW15)
- 16 user LEDs (LD0-LD15)
- 5 user pushbuttons (BTNL,BTNR,BTNU,BTND,BTNC)
- 4-digit 7-segment display (DISP1)
- Three Pmod ports (JA,JB,JC)
- Pmod for XADC signals (JXADC)
- 12-bit VGA output (VGA)
- USB-UART Bridge
- Serial Flash
- USB-JTAG port for FPGA programming and communication (J4)
- USB HID Host for mice, keyboards and memory sticks (J2)

5. Asynchronous reset

In this lab, all your modules must use asynchronous, active-low, reset signal. This type of reset is a common approach in practice, due to 2 main reasons:

1. High-to-low transition: Most logic families can sink more current (to GND) than they can source (to VDD). The reset signal is required simultaneously in many places in chip, which requires a high fan-out. Therefore, the ability to transit from high to low is preferable.
2. Asynchronous signal forces an immediate simultaneous reset through the chip. This reset is regardless of the clock, which might be in different modes (toggling or idle) in different clock/power domains in the chip. Whereas in the synchronous reset case, the first condition to begin a reset is that the clock will reach its active edge.

One must keep in mind that asynchronous reset is prone to metastability issues, therefore an asynchronous reset must be provided for a long enough time.

To avoid confusion, the active low asynchronous reset port is named **rstn**.

6. PS2 interface

The PS2 protocol is a simple two-wire scheme that uses serial transmission to transmit the data to the board. On our board, this 2-wire PS2 communication from the USB port is provided to FPGA pins C17 and B17 by the USB-HID controller, as depicted in Figure 6. More information on the PS2 protocol can be found [here](#) and in the Basys3 reference manual.

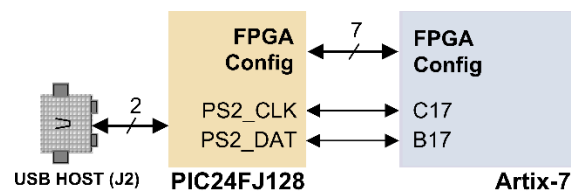


Figure 6 - BASYS3 board connections from FPGA to the USB port

From the moment user presses (and not yet releases) a key, the keyboard transmits a sequence of bits called "make-code". This sequence is carrying a special 1-Byte scan-code of the keyboard key, and each key on the keyboard is given a unique "scan-code", see 7. If the user is holding the key, the "make-code" packet is retransmitted every 100ms.

When a key is released, the keyboard transmits a 2-Byte "break code" sequence consisting of the first byte being 0xF0, followed by the 2nd byte being the scan-code of the key that was released.

For example, consider the following scenario, where user presses the letter 'a':

1. User presses the 'a' key
2. Keyboard sends "make code" (which is 0x1C for the 'a' key) serially. The keyboard keeps sending the "make code" every 100ms until the user releases the key.
3. User releases the 'a' key
4. Keyboard sends the "key up" code 0xf0 serially
5. Keyboard sends the "break code" (which is 0x1C for the 'a' key) serially

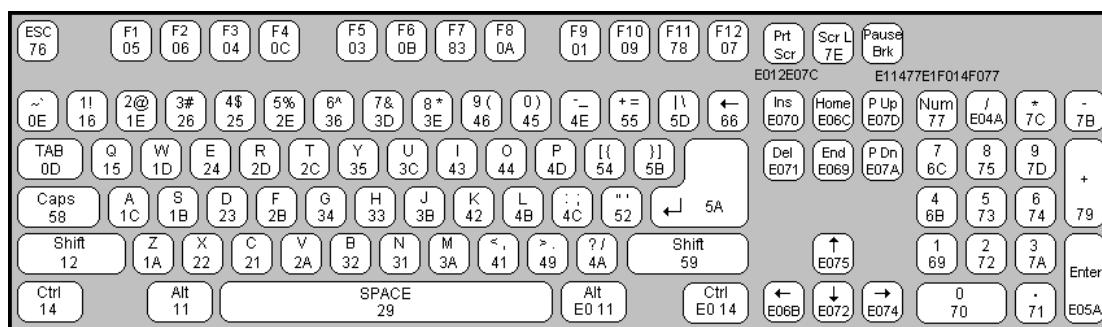


Figure 7 - Keyboard keys and their scan-codes in hexadecimal representation.

To transmit the sequence of bytes, the keyboard first forces the *DATA* line low to create the start bit. Bits are transmitted using the **negative edge of CLK** for synchronization. This is illustrated in Figure 8. The *DATA* signal changes state when the *CLK* signal is high, and *DATA* is valid for reading on the falling edge of *CLK*. This *CLK* signal is slow, 20-30KHz, and this clock is kept constantly high when idle. **Namely, this clock doesn't always toggle!**

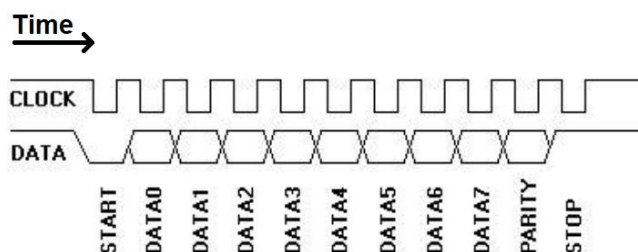


Figure 8 - Timing diagram of PS2 data and clock signals

The basic operation of a PS2 interface is as follows:

1. On the falling edge of *CLK*, use a shift register to capture each bit of data.
2. When enough bits have been sent (i.e. when start bit and scan-code bits are captured and the parity bit is on the PS2Data line, no need to wait for the deterministic stop bit), you can look at the scan-code and decide what to do:
 - 2.1. If the received Byte is neither an extended key qualifier (0xE0) nor a “key up” (0xF0), you know that this is a key-pressed event indication.
 - 2.2. If the received byte is “key up” (0xF0) then the following packet will tell the scan-code of the key that was released

For example, when “CAPS-lock” key is released, and your design receives the incoming bits into a 22-bit long shift register (input to the shift-register is better be through its **MSB**²), then after 22 clock cycles the following content will be in the shift register:

From Ps2Data	R[21]	R[20]	R[19]	R[18]	R[17]	R[16]	R[15]	R[14]	R[13]	R[12]	R[11]	R[10]	R[9]	R[8]	R[7]	R[6]	R[5]	R[4]	R[3]	R[2]	R[1]	R[0]
	1	0	0	1	0	1	1	0	0	0	0	1	1	1	1	1	1	0	0	0	0	0
	Stop	Par	Data[7:0]=0x58								Start	Stop	Par	Data[7:0]=0xF0								Start

Important Information

1. The PS2 uses a simple error detection scheme called **Odd Parity**. The sender asserts the parity bit value such that the total number of high bits in data and parity

² Maintaining the shift-register in this manner (pushing new bit into the MSB) helps you in receiving the packets in a correct order for a convenient further processing. If you'd push new bits into the LSB of the shift register, you will probably need to reverse the content of this register.

together is odd. When the receiver receives the data frame, he checks odd-parity condition to detect 1/3/5/7/9-bit flip errors.

2. Some keys on the keyboard (like the arrow keys) are special in the sense that they send an additional code “E0” ahead of the scan code. Such keys are called extended keys. When an extended key is released and “E0 F0” code is sent followed by the scan code. So, regardless of the key, the last two chunks of data when a key is released will be “F0” and the scan code.
3. When your keyboard will be connected to the BASYS3 board, **no lights should be lit on the keyboard**. The keyboard LEDs, such as caps-lock, are lit after the host PC sends messages to keyboard, however in our lab you are not supposed to send anything to the keyboard. But you should be able seeing an orange led blinking next to the USB port on the BASYS board.
4. Although the keyboard clock and data wires are bidirectional, we will only be using it as inputs for this lab. [Typically writing to the keyboard is used to reset, turn on the various indicator lights, etc.] The bidirectionality of the line is provided by the **open-collector** driver at each terminal connected to the wire.
5. Constraints file: you will need to enable the following pins (by uncommenting them and verifying that they are mapped to correct IO ports in your Verilog top module):

FPGA pin names	IO port name (from top module in Verilog)	Description
U18	btnC (for reset)	for the active-high reset
U2, U4, V4, W4	an[0:3]	Active-low digit selection (7-segment)
V7	dp	Active-low dot (7-segment)
W7,W6,U8,V8,U5,V5,U7	seg[0:6]	Active-low 7 segments (7-segment)
U16, E19, U19	led[0:2]	Active high user signals to the LEDs
W5	clk	System clock 100MHz
C17, B17	PS2Clk, PS2Data	PS2 protocol communication lines (fed to FPGA from USB port via HID controller)

6. The 2 signals PS2Clk and PS2Data require the 3rd (pull-up-enable) constraint line to be uncommented for them. The purpose is to drive the high-Z output of keyboard into a logical 1.

```
set_property PULLUP true [get_ports PS2Data]
set_property PULLUP true [get_ports PS2Clk]
```
7. In case you are getting an error related to the keyboard clock during the Place & Route step in Xilinx ISE (which says something like "Clock IOB/ clock component is

not..."), please add the following line to your XDC file and then re-run the Place & Route step again:

```
set_false_path -from [get_ports PS2Clk] -to [get_clocks sys_clk_pin]
```

where *PS2Clk* is the name of keyboard clock signal in your design.

8. VGA interface

A VGA monitor scans the screen row by row, starting at the upper left corner and ending at the lower-right corner. This scan order also known as Raster-scan. The scan is synchronized using two signals, called *Hsync* (horizontal synchronization), and *Vsync* (vertical synchronization). The *Hsync* signal is normally high and it goes low to signal the end of the line scan. The *Vsync* signal is normally high and it goes low to signal the end of the whole screen scan, meaning that the scan should re-start back in the upper left corner of the screen. In addition to these synchronization signals, the VGA screen is fed with the color of the current pixel, which consists of red, green and blue color channels.

Screen Synchronization

We will focus on a 800×600 pixel screen display. A pixel clock will be 50 MHz. To get 800 pixels horizontally, a horizontal synchronization frequency of approximately 48.07 KHz is required. This corresponds to approximately 1040 clock periods of the pixel clock. During the first 800 clock periods of the pixel clock for a row, visible pixels can be displayed; however, the last 240 clock periods for the row are called the “retrace period” or “blanking region” where nothing is displayed while the beam retraces back to the left of the screen. To get 600 pixels vertically, a vertical synchronization frequency of 72 Hz is required. This corresponds to approximately $(1040 \times) 666$ clock periods of the pixel clock. This can be thought of as generating 600 visible rows followed by 66 blank rows during which the beam retraces back to the top of the screen. This is illustrated in Figure. A summary of numbers of the pixel clocks for each region can be found [here](#).

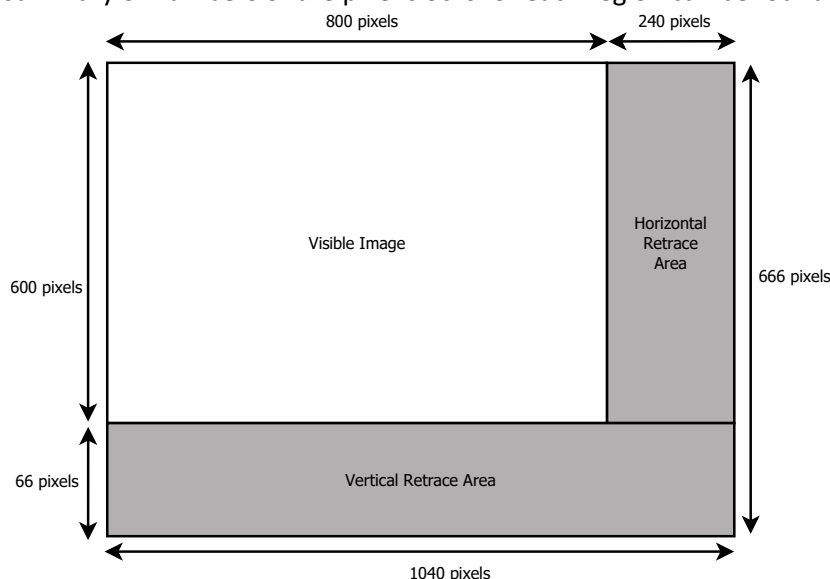


Figure 9 - VGA standard display regions for 800x600 resolution

Figure shows the timing of the *Hsync* signal. It is made low starting on the 856th pixel clock period for the row and made high again on the 976th pixel clock period. During the first 800 pixel-clock periods, visible pixels are generated. During the last 240 pixel-clock

periods, nothing is generated on the screen, and a black color is expected from the pixel source.

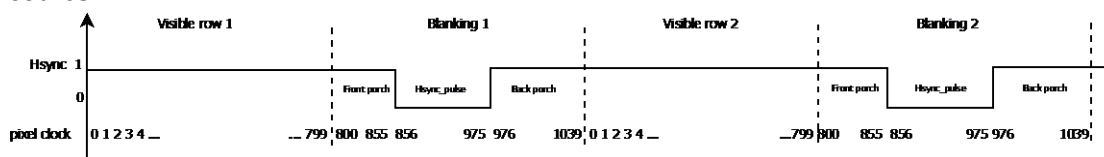


Figure 10 - Horizontal sync timing example

Figure shows the relationship between the Vsync signal and the Hsync signal. The digits represent the line count. As stated earlier, the first 600 lines are displayed while the last 66 lines correspond to the retrace period for the beam to get back to the top left corner of the screen.

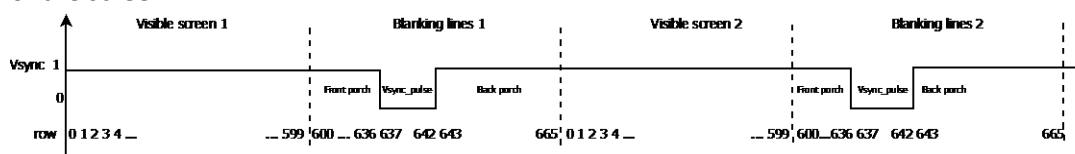


Figure 11 - Vertical sync timing example

Colors: The VGA connector is analog, meaning that the red, green and blue colors are must be expressed by continuous voltages on 3 separate wires. However the Artix-7 FPGA chip has digital output pins dedicated for the R,G,B colors (namely, each color is expressed by 4 bits). This gives a total of 16 values levels per color-channel, and a total of 4096 different colors possible to output. To convert the digital color channels to analog ones, a simple DAC circuit is implemented on the Basys3 board, which takes the digital bits and performs a weighted sum of voltages. See Figure 12 for illustration.

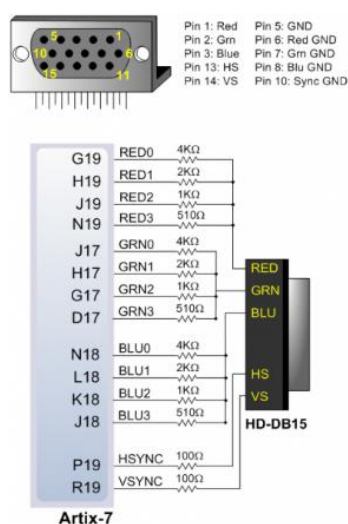


Figure 12 - The conversion of the digital FPGA signals into analog signals for the VGA connector

In this lab, you may find [this table](#) useful to determine the digital RGB color codes. Pay attention that you don't have 256 (8b) but only 16 (4b) levels per channel in your implementation. Think about ways to properly down-sample the 8bits to 4bits color code.

Constraints file: to connect the design to the VGA pins, go to you xdc file and uncomment the lines that define the pins that appear in Table 2. Note that additional pins should be enabled to receive a clock, reset etc.

FPGA Pin names	Verilog output port names
G19,H19,J19,N19	vgaRed[0:3]
N18,L18,K18,J18	vgaBlue[0:3]
J17,H17,G17,D17	vgaGreen[0:3]
P19	Hsync
R19	Vsync

Table 2 - Artix-7 VGA related pin names